QISKIT EXPERIMENTS VIVA QUESTION BANK

Detailed, easy-to-say answers for Experiments 2-9, Case Study 10, and setup checks

Concept questions | source-code questions | predicted outputs | step-by-step traces | common viva traps | mock viva

How to practice: First answer the quick questions in your own words. Then use the expanded responses as a model: name the initial state, explain each gate or code step, predict the measurement, and mention what could change on real hardware. Items tagged **[General knowledge]** extend beyond the supplied source.

Based on the supplied cqi.md notebook transcription. Its code snippets were not executed against Qiskit or an IBM Quantum account. Secrets from the source are not reproduced.

Contents

Experiment 2 - Statevectors and Sampling 3

Experiment 3 - Identity Matrices 5

Experiment 4 - Pauli X, Y, and Z Gates 7

Experiment 5 - Hadamard Gate 9

Experiment 6 - SWAP and CNOT 11

Experiment 7 - Toffoli / CCX Gate 13

Experiment 8 - Bell-State Circuits 15

Experiment 9 - CCX with Both Controls Set 17

Case Study 10 - Bell Circuit on IBM Quantum Hardware 19

Environment and Simulator Checks 21

Terminology follows the notebooks: q0/q1 identify qubits; c0/c1 identify classical bits. In Qiskit count strings, c0 is printed at the right.

Experiment 2 - Statevectors and Sampling

Program focus: Statevector([a,b]) stores amplitudes for |0> and |1>. The source defines u, v, normalized w, draws states, samples v for 1000 shots, and plots counts.

Quick viva questions

Q1. What is a qubit statevector?

Easy answer. It is a list of complex amplitudes describing a pure quantum state. For one qubit, [alpha,beta] means alpha|0> + beta|1>, and a valid state has |alpha|^2+|beta|^2=1.

Q2. What does Statevector([1/sqrt(2),1/sqrt(2)]) represent?

Easy answer. It represents |+> = (|0>+|1>)/sqrt(2). Measuring in the computational basis gives 0 and 1 with probability one-half each.

Q3. Find the probabilities for v=[(1+2i)/3,-2/3].

Easy answer. The |0> amplitude has squared magnitude (1^2+2^2)/9=5/9; the |1> amplitude has squared magnitude 4/9. The probabilities add to 1.

Q4. Why was w changed to [1/sqrt(5),2/sqrt(5)]?

Easy answer. A valid statevector must have total probability 1. The original [1/3,2/3] had squared norm 5/9, so it was not normalized. The corrected values keep the amplitude ratio and have squared magnitudes 1/5 and 4/5.

Q5. What does sample_counts(1000) return?

Easy answer. It draws 1000 simulated measurement outcomes according to the state probabilities and returns a counts dictionary. For v, counts should be near 556 zeros and 444 ones, but the exact values vary randomly.

Q6. Why may the histogram not show exact theoretical probabilities?

Easy answer. A finite number of samples has statistical variation. More shots usually make the observed fractions closer to the theoretical probabilities, but do not make every finite result exact.

Q7. What is the difference between an amplitude and a probability?

Easy answer. An amplitude can be complex and can have a phase. Measurement probability is the squared magnitude of the amplitude, so phase does not change a single computational-basis probability.

Q8. What does draw("latex") or draw("text") do?

Easy answer. It displays a representation of the state for inspection. Drawing is visualization; it does not perform a measurement or change the mathematical state.

Q9. what is normalization? [General knowledge]

Easy answer. Normalization means the squared magnitudes of all amplitudes sum to one, ensuring total measurement probability is one.

Experiment 2 - Statevectors and Sampling - expanded answers

Practice question 1. Explain the statevector and the three states in this experiment.

Suggested spoken answer. A one-qubit statevector stores the amplitude of |0> followed by the amplitude of |1>. State u has equal positive amplitudes, so it is an equal superposition. State v has probabilities 5/9 and 4/9. State w is normalized and has probabilities 1/5 and 4/5.

Practice question 2. Explain the sampling result for v.

Suggested spoken answer. The program asks for 1000 samples. Since v gives probability 5/9 for zero and 4/9 for one, I expect roughly 556 zeros and 444 ones. The histogram shows an estimate from random samples, so the exact counts can differ.

Practice question 3. Why is the correction to w important?

Suggested spoken answer. Quantum amplitudes must satisfy the normalization rule. The original vector had squared magnitudes summing to 5/9, so it could not represent a normalized physical pure state. Dividing the same amplitude ratio by sqrt(5) gives a valid vector and probabilities 1/5 and 4/5.

Experiment 3 - Identity Matrices

Program focus: identity_matrix(n_qubits) creates a circuit, applies id() to each qubit, converts it to Operator, and returns the matrix data.

Quick viva questions

Q1. What does an identity gate do?

Easy answer. It leaves the quantum state unchanged. Its matrix has ones on the main diagonal and zeros elsewhere.

Q2. What matrix size is expected for n qubits?

Easy answer. A system of n qubits has 2^n computational basis states, so its operator matrix is 2^n by 2^n. For one, two, and three qubits, the sizes are 2x2, 4x4, and 8x8.

Q3. Why does the function loop over each qubit?

Easy answer. It applies an identity operation to each individual wire in the circuit. Together these operations still leave the complete register unchanged.

Q4. What does Operator(qc).data provide?

Easy answer. It converts the circuit's unitary action into a numerical matrix representation that can be printed or inspected.

Q5. Why reject a negative qubit count?

Easy answer. A quantum register cannot contain a negative number of qubits. The explicit ValueError gives a clear response before circuit construction.

Q6. What does n_qubits=0 mean in this implementation?

Easy answer. The validation allows zero. A zero-qubit circuit corresponds to a one-dimensional identity operator, although it is not a typical lab example.

Q7. What is a useful check for an identity matrix?

Easy answer. It must be square, have ones on the diagonal, zeros off the diagonal, and multiply any state by leaving it unchanged.

Q8. what does an identity gate do to a superposition? [General knowledge]

Easy answer. It preserves both the amplitudes and their relative phases, so the complete state is unchanged.

Experiment 3 - Identity Matrices - expanded answers

Practice question 1. Explain the identity-matrix experiment.

Suggested spoken answer. The function creates an n-qubit circuit and applies an identity gate to every qubit. An identity gate changes nothing, so converting the circuit into an Operator produces the identity matrix for the whole register. For n qubits, that matrix has 2^n rows and 2^n columns.

Practice question 2. What should the outputs for one, two, and three qubits look like?

Suggested spoken answer. For one qubit, the result is a 2x2 identity matrix; for two qubits it is 4x4; for three it is 8x8. In every case only the main diagonal contains ones. This exponential matrix growth is why printing larger examples quickly becomes impractical.

Practice question 3. What boundary check is in the supplied function?

Suggested spoken answer. It raises a ValueError if n_qubits is negative. Zero is allowed and represents a one-dimensional identity operator, though the main examples use one to three qubits.

Experiment 4 - Pauli X, Y, and Z Gates

Program focus: Each circuit has one qubit initialized to |0>, applies one Pauli gate, then measures into one classical bit.

Quick viva questions

Q1. What does Pauli-X do?

Easy answer. It exchanges |0> and |1>, so it is the quantum bit-flip gate. Applied to the circuit's initial |0>, it produces |1> and measurement returns 1.

Q2. What does Pauli-Y do to |0>?

Easy answer. With the standard convention, Y|0>=i|1>. The phase i is present in the state, but computational-basis measurement still returns 1 with certainty.

Q3. What does Pauli-Z do to |0>?

Easy answer. Z|0>=|0>, so this circuit measures 0. Z changes the phase of |1> but not |0>.

Q4. Why can Y and X give the same measurement count here?

Easy answer. Both send the initial |0> to a state whose only computational-basis outcome is 1. Their resulting states differ by a phase, but this measurement does not reveal that phase.

Q5. What is a Pauli gate generally used to illustrate?

Easy answer. The three basic Pauli matrices represent bit flip (X), a bit-and-phase transformation (Y), and phase flip (Z). They are common one-qubit operations.

Q6. What does measure(qreg[0],creg[0]) mean?

Easy answer. It measures qubit 0 and stores the classical result in classical bit 0.

Q7. Why does the circuit create both quantum and classical registers?

Easy answer. The quantum register holds the state being transformed; the classical register stores the classical bit produced by measurement.

Q8. are Pauli gates reversible? [General knowledge]

Easy answer. Yes. Each Pauli gate is unitary and is its own inverse: applying it twice returns the original state.

Experiment 4 - Pauli X, Y, and Z Gates - expanded answers

Practice question 1. Explain the expected result of each Pauli circuit.

Suggested spoken answer. All three circuits start in |0>. X changes it to |1>, so the measurement is 1. Y changes it to i|1>, which also measures as 1 because the phase does not alter the probability. Z leaves |0> unchanged, so its measurement is 0.

Practice question 2. Why is it incomplete to say X and Y are the same gate?

Suggested spoken answer. They happen to produce the same outcome in this particular measurement, but their quantum states differ by a phase. That phase can affect later interference if more gates are applied before measurement.

Practice question 3. Explain measurement and the two registers.

Suggested spoken answer. The quantum register stores the qubit state and receives the gate operation. Measurement converts the quantum result into a classical bit, and the explicit mapping stores qreg[0] into creg[0].

Experiment 5 - Hadamard Gate

Program focus: A one-qubit circuit applies H to qubit 0 and measures it into classical bit 0.

Quick viva questions

Q1. What is the Hadamard gate?

Easy answer. It creates and removes equal superpositions. In particular H|0>=(|0>+|1>)/sqrt(2), while H|1>=(|0>-|1>)/sqrt(2).

Q2. What result should repeated measurements produce?

Easy answer. Ideally, zero and one each occur about half the time because the state has equal squared amplitudes.

Q3. Does a single measurement show both values?

Easy answer. No. Each individual shot returns one classical result, either zero or one. The probability distribution is revealed by repeating the experiment.

Q4. What does qc.h(0) do?

Easy answer. It applies the Hadamard operation to qubit 0 in the circuit.

Q5. Why is a measurement needed to see counts?

Easy answer. Before measurement, the state is a coherent superposition. Measurement produces a classical outcome that can be counted.

Q6. is superposition the same as a 50/50 classical mixture? [General knowledge]

Easy answer. Not generally. A superposition has phase relationships that can affect later interference; a classical mixture does not preserve those coherent phase relationships.

Experiment 5 - Hadamard Gate - expanded answers

Practice question 1. Explain the Hadamard experiment in spoken form.

Suggested spoken answer. The qubit begins in |0>. The Hadamard gate turns it into an equal superposition of |0> and |1>. When I measure once, I only see one result, but if I repeat the circuit many times I expect roughly half zeros and half ones.

Practice question 2. What is the role of repeated shots?

Suggested spoken answer. A shot is one execution and measurement of the circuit. Repeating it estimates the probability distribution; finite samples fluctuate around the ideal 50/50 result.

Practice question 3. How is a superposition different from a classical random choice?

Suggested spoken answer. Both can produce random-looking measurement outcomes, but a quantum superposition retains phase information. That phase can later create constructive or destructive interference, which a classical mixture cannot do.

Experiment 6 - SWAP and CNOT

Program focus: Two-qubit circuits apply either swap(0,1) or cx(0,1), then measure both qubits into classical bits with matching indices.

Quick viva questions

Q1. What does SWAP do?

Easy answer. It exchanges the states of two qubits. It leaves |00> and |11> unchanged, and exchanges |01> with |10>.

Q2. What does CNOT do?

Easy answer. CNOT flips its target if and only if its control is 1. In qc.cx(0,1), qubit 0 is the control and qubit 1 is the target.

Q3. What happens in these exact source circuits starting from |00>?

Easy answer. Both SWAP and CNOT leave |00> unchanged, so both circuits measure 00 every ideal shot. To demonstrate their difference, the input needs a one on an appropriate qubit.

Q4. How would SWAP act on |01>?

Easy answer. It exchanges the two qubit values and produces |10>.

Q5. How would CNOT act when control q0 is 1 and target q1 is 0?

Easy answer. It flips q1, so the logical state becomes q0=1,q1=1.

Q6. What is a common Qiskit bit-ordering mistake?

Easy answer. In displayed bit strings, classical bit 0 is the rightmost character. A two-bit result is written c1c0, even though measurements are mapped in the order [q0,q1] to [c0,c1].

Q7. why is CNOT important? [General knowledge]

Easy answer. Together with suitable one-qubit gates, CNOT can create entanglement and is a standard two-qubit building block.

Experiment 6 - SWAP and CNOT - expanded answers

Practice question 1. Compare SWAP and CNOT.

Suggested spoken answer. SWAP exchanges the two qubit states regardless of their values. CNOT keeps the control unchanged and flips only the target when the control is one. In the supplied circuits both receive |00>, so both appear to do nothing; a test input such as |01> distinguishes them.

Practice question 2. Explain the control and target in the supplied CNOT.

Suggested spoken answer. The call cx(0,1) means qubit 0 controls the operation and qubit 1 is the target. If q0 is zero, q1 is unchanged; if q0 is one, q1 is flipped. This order matters, because reversing the arguments creates a different operation.

Practice question 3. How should the measured bit string be read?

Suggested spoken answer. The mapping sends qubit 0 to classical bit 0 and qubit 1 to classical bit 1. Qiskit prints bit 1 on the left and bit 0 on the right, so the string is c1c0. I should state that convention before tracing a result.

Experiment 7 - Toffoli / CCX Gate

Program focus: A three-qubit circuit applies ccx(0,1,2) to the all-zero initial state and measures all three qubits.

Quick viva questions

Q1. What is a Toffoli gate?

Easy answer. It is a doubly controlled NOT gate. The first two qubits are controls, and the third is the target.

Q2. When does CCX flip its target?

Easy answer. Only when both control qubits are 1. If either control is zero, the target is unchanged.

Q3. What is the output of this exact circuit?

Easy answer. It starts in |000>, so both controls are zero and the target is not flipped. The measured result is 000.

Q4. Give an input that makes the target flip.

Easy answer. An input with both controls set, such as q0=1 and q1=1, flips q2 from zero to one.

Q5. Why is Toffoli useful conceptually?

Easy answer. It demonstrates a reversible multi-qubit logic operation with two controls.

Q6. why is reversible logic useful in quantum computing? [General knowledge]

Easy answer. Quantum gates must be reversible, represented by unitary operations; irreversible classical logic needs to be embedded into reversible operations when used coherently.

Experiment 7 - Toffoli / CCX Gate - expanded answers

Practice question 1. Explain how the Toffoli gate works.

Suggested spoken answer. The CCX gate has two control qubits and one target qubit. It checks whether both controls are one; only in that case does it flip the target. The controls themselves are not changed.

Practice question 2. Trace the supplied circuit.

Suggested spoken answer. All qubits begin at zero. The circuit applies CCX with q0 and q1 as controls and q2 as target. Since both controls are zero, the gate does nothing, so the measurement is 000.

Practice question 3. How can you change the input so CCX visibly acts?

Suggested spoken answer. Apply X gates to both control qubits before CCX. Then both controls are one, so the target flips from zero to one. That is exactly the logic shown in Experiment 9.

Experiment 8 - Bell-State Circuits

Program focus: Two separate circuits compare H(q0) then CX(q0,q1) with CX first then H(q0); both measure q0,q1 into c0,c1.

Quick viva questions

Q1. How does H followed by CNOT create a Bell state?

Easy answer. H on q0 creates (|00>+|01>)/sqrt(2) in q1q0 ket ordering; CNOT uses q0 as control and flips q1 in the |01> branch, producing (|00>+|11>)/sqrt(2).

Q2. What measurement counts are expected from the Bell circuit?

Easy answer. Ideally, only 00 and 11 appear, with roughly equal counts. The outcomes are correlated even though either single qubit alone looks random.

Q3. What happens when CNOT is applied first to |00>?

Easy answer. It does nothing because q0, the control, is zero. H on q0 then creates a product superposition, not an entangled Bell state.

Q4. What outcomes does the CNOT-then-H circuit produce?

Easy answer. It produces q0 random and q1 fixed at zero. With Qiskit's printed c1c0 ordering, the outcomes are 00 and 01, about equally often.

Q5. Are the two circuits equivalent?

Easy answer. No. Gate order matters: H then CNOT creates entanglement; CNOT then H on the control leaves a separable state.

Q6. What is entanglement in this example?

Easy answer. The two-qubit Bell state cannot be written as a product of independent one-qubit states. Measuring one qubit predicts the other in the computational basis.

Q7. what does a Bell inequality test? [General knowledge]

Easy answer. It tests correlations that cannot be explained by local hidden-variable models; the supplied circuits demonstrate Bell-state preparation, not a Bell inequality experiment.

Experiment 8 - Bell-State Circuits - expanded answers

Practice question 1. Explain the Bell-state circuit.

Suggested spoken answer. The circuit starts in |00>. H on q0 creates an equal superposition of q0 being zero and one. Then CNOT uses q0 as control: it flips q1 only in the q0=1 branch. The result is an entangled state with equal amplitudes for 00 and 11.

Practice question 2. Explain the expected counts and correlation.

Suggested spoken answer. On repeated ideal measurements, the Bell circuit produces 00 and 11 about half the time each, and not 01 or 10. Each single bit is individually unpredictable, but the two bits always agree.

Practice question 3. Why does reversing the gate order change the result?

Suggested spoken answer. CNOT first sees q0=0, so it does nothing. The Hadamard then puts q0 into a superposition while q1 stays zero. The outcomes are 00 and 01 in printed Qiskit order, so the qubits are not entangled.

Experiment 9 - CCX with Both Controls Set

Program focus: X gates set q0 and q1 to one, CCX(0,1,2) flips q2, and all qubits are measured.

Quick viva questions

Q1. What does each X gate do before CCX?

Easy answer. X on q0 changes it from zero to one; X on q1 does the same. The target q2 remains zero before the controlled operation.

Q2. Why does the Toffoli now flip q2?

Easy answer. Both controls are one, so CCX toggles target q2 from zero to one.

Q3. What is the final measurement?

Easy answer. All three qubits are one, so the ideal result is 111.

Q4. What is the difference between Experiments 7 and 9?

Easy answer. Experiment 7 starts with controls 00, so CCX does not flip the target. Experiment 9 prepares both controls as one, so the target flips.

Q5. How many gates are in the unitary portion of this circuit?

Easy answer. Three: X on q0, X on q1, and one CCX. Measurements follow those gates.

Q6. is this a quantum speedup demonstration? [General knowledge]

Easy answer. No. It demonstrates gate behavior and conditional logic, not a computational speedup.

Experiment 9 - CCX with Both Controls Set - expanded answers

Practice question 1. Trace the CCX circuit step by step.

Suggested spoken answer. Initially the state is 000. The first X changes q0, giving q0=1; the second X changes q1, so both controls are now one while q2 remains zero. CCX therefore flips q2 and the final state is 111.

Practice question 2. Compare this experiment with the previous Toffoli circuit.

Suggested spoken answer. The gate is the same, but the input differs. With zero-valued controls, as in Experiment 7, the target does not change. Here the X gates set both controls to one, so the target is flipped.

Practice question 3. What result should the measurement histogram show?

Suggested spoken answer. On an ideal simulator or noiseless execution, every shot should be 111. Real hardware can show a small number of other outcomes due to gate and readout errors.

Case Study 10 - Bell Circuit on IBM Quantum Hardware

Program focus: The code loads credentials from environment variables, selects an operational non-simulator backend, transpiles the Bell circuit for it, submits with SamplerV2, and reads counts.

Quick viva questions

Q1. Why transpile a circuit before running on hardware?

Easy answer. A real backend has a specific set of supported operations, qubit connectivity, and constraints. Transpilation rewrites the circuit into operations and qubit layout the selected backend can execute.

Q2. What does least_busy(operational=True, simulator=False) request?

Easy answer. It asks the service for an operational physical backend among those available, selecting a relatively less busy one at that time.

Q3. What is a shot?

Easy answer. One execution of the circuit followed by measurement. With 1024 shots, the result estimates the Bell-state output distribution over 1024 executions.

Q4. What is SamplerV2 used for?

Easy answer. It submits circuits for sampling and returns shot-based measurement results. The code obtains the result for the first submitted circuit and reads counts from its classical register c.

Q5. Why can hardware counts differ from 50/50?

Easy answer. Finite sampling creates random variation, and real devices also have gate, readout, and environmental noise.

Q6. What does the Job ID do?

Easy answer. It identifies a submitted runtime job so it can be tracked or retrieved later.

Q7. Why use environment variables for credentials?

Easy answer. They keep secret values out of the source code and shared notebook text. Access should still be protected, and exposed credentials should be revoked or rotated.

Q8. What does optimization_level=1 mean here?

Easy answer. It selects a preset transpilation optimization setting. The transpiler balances circuit rewriting effort and optimization; the backend argument supplies target constraints.

Q9. why is the hardware result not guaranteed to match an Aer simulation? [General knowledge]

Easy answer. Aer can simulate an ideal or configured noise model; physical hardware has device-specific noise, calibration changes, and sampling variation.

Case Study 10 - Bell Circuit on IBM Quantum Hardware - expanded answers

Practice question 1. Explain the hardware execution workflow.

Suggested spoken answer. First the code connects to the IBM Quantum service and chooses an available operational hardware backend. It builds the Bell circuit, then transpiles it so the circuit fits that backend. SamplerV2 runs the transpiled circuit for 1024 shots, and the program reads and plots the measurement counts.

Practice question 2. Why do we need transpilation and sampling?

Suggested spoken answer. Transpilation adapts abstract gates and connectivity to the selected device. Sampling repeats the circuit and estimates output probabilities; it does not return a single exact probability distribution. On ideal hardware the Bell results are 00 and 11, but real counts can be affected by noise.

Practice question 3. Explain the credential handling safely.

Suggested spoken answer. The program reads credentials from environment variables instead of writing them directly in the notebook. That reduces accidental exposure when sharing code, but the environment itself must remain private. If a token has been exposed, it should be revoked or rotated before further use.

Environment and Simulator Checks

Program focus: The separate test notebook prints package versions, builds/draws a Bell circuit, runs AerSimulator, retrieves counts, and plots a histogram.

Quick viva questions

Q1. Why print package versions?

Easy answer. It helps identify which Python and library versions are active when reproducing results or debugging compatibility problems.

Q2. What does AerSimulator do?

Easy answer. It simulates a quantum circuit on a classical computer and returns measurement samples. It is useful for local tests without submitting a job to quantum hardware.

Q3. What is the expected Bell simulation result?

Easy answer. For H(q0) then CX(q0,q1), ideal simulation yields 00 and 11 with approximately equal counts over many shots.

Q4. Why can simulator counts vary from run to run?

Easy answer. Sampling is random unless a seed is set. The probabilities are fixed for an ideal circuit, but finite-shot counts fluctuate.

Q5. What is the difference between qc.draw and qc.draw("mpl")?

Easy answer. Both visualize the circuit; the mpl option uses Matplotlib to render a graphical diagram. Drawing itself does not execute the circuit.

Q6. What is the difference between simulation and hardware execution?

Easy answer. Simulation uses a classical computer model; hardware uses physical qubits and can show noise, queueing, and device-specific limitations.

Q7. What is the purpose of checking Python and IPython versions?

Easy answer. It helps confirm the notebook kernel and runtime environment, especially when imports or visualizations fail.

Q8. What does the source say about execution status?

Easy answer. The code was transcribed and reviewed but not run against a Qiskit installation or IBM account, so it should not be described as verified execution.

Q9. what is a random seed? [General knowledge]

Easy answer. A seed initializes a pseudo-random number generator so repeated simulation sampling can be reproducible when the simulator supports seeded randomness.

Environment and Simulator Checks - expanded answers

Practice question 1. Explain the local Bell simulation.

Suggested spoken answer. The notebook builds H on q0 followed by CNOT from q0 to q1, then adds measurements. AerSimulator runs this circuit locally for a chosen number of shots and returns counts. For an ideal simulation, the results should be concentrated on 00 and 11 in roughly equal proportions.

Practice question 2. What can the setup checks tell you?

Suggested spoken answer. The printed versions tell you which interpreter and libraries are being used. The circuit drawing confirms the intended gate order visually, while simulation checks the behavior locally without an IBM hardware job.

Practice question 3. What should you say about validation?

Suggested spoken answer. The source notes that the cells were transcribed and reviewed but were not executed against Qiskit or an IBM account. Therefore I can explain the expected behavior, but I should not claim that this exact notebook run was verified.

Qiskit Bit-String Reminder

Qiskit labels classical bit 0 as the rightmost character in the displayed result. In the supplied measurements, q0 is mapped to c0 and q1 to c1, so a two-bit string is printed as c1c0. Always state this before interpreting a count key.

Examples for the supplied circuits

Circuit/input	Expected ideal count keys	Reason
X on initial |0>	1	X flips q0.
Z on initial |0>	0	Z leaves |0> unchanged.
H(q0), CX(q0,q1)	00 and 11	Bell state; bits agree.
CX(q0,q1), H(q0)	00 and 01	q0 varies; q1 stays 0.
X(q0), X(q1), CCX(q0,q1,q2)	111	Both controls are 1; target flips.

Rapid Revision

Topic	Recall
Statevector	Amplitudes are complex; probabilities are squared magnitudes; normalize to 1.
Sampling	Finite-shot counts fluctuate around probabilities; more shots improve estimates.
Identity	n qubits: 2^n by 2^n identity matrix.
Paulis	X flips bit; Y flips with phase; Z flips phase.
Hadamard	H|0>=(|0>+|1>)/sqrt(2).
CNOT	First argument control, second target.
Toffoli	Target flips only if both controls are 1.
Bell	H then CNOT creates correlated entangled state.
Hardware	Transpile for backend; run sampler; hardware noise affects counts.
Aer	Local simulation, not a physical quantum device.

Mock Viva

Try answering aloud before reading each checkpoint.

1. A state has amplitudes 1/sqrt(2),1/sqrt(2). What is the measurement distribution?

Checkpoint. Half zero and half one in the computational basis.

2. Why are counts from 1000 shots not exact probabilities?

Checkpoint. Sampling variation makes finite counts fluctuate.

3. What does H followed by CNOT do to |00>?

Checkpoint. Creates the Bell state (|00>+|11>)/sqrt(2).

4. What is the output of the supplied CNOT-then-H circuit?

Checkpoint. q0 is random and q1 remains zero; count keys 00 and 01.

5. Why do both supplied SWAP and CNOT circuits return 00?

Checkpoint. Both start in |00>, which each gate leaves unchanged.

6. How do Experiments 7 and 9 differ?

Checkpoint. Experiment 9 sets both CCX controls with X gates, so the target flips.

7. What does transpilation adapt?

Checkpoint. The circuit's gates and layout to a selected backend's constraints.

8. What is one difference between Aer and hardware?

Checkpoint. Aer simulates; hardware has device noise and physical execution.

9. Why does Qiskit print c0 on the right?

Checkpoint. Its displayed bit strings put the least-significant bit, bit zero, at the right.

10. Does the notebook have a verified run?

Checkpoint. No; the supplied transcription explicitly says it was not run.

Viva answer checklist

Name the input state; state which qubit each gate acts on; apply gates in order; convert amplitudes to probabilities; explain how measurement displays the result; distinguish ideal simulation from noisy hardware.

References and source notes

Primary source: supplied cqi.md, which transcribes the experiment notebooks and identifies corrections. The guide preserves that separation between numbered experiments, the IBM Quantum case study, and the independent environment/simulator checks.

Qiskit bit ordering
https://docs.quantum.ibm.com/guides/bit-ordering

Qiskit Statevector API
https://docs.quantum.ibm.com/api/qiskit/2.0/qiskit.quantum_info.Statevector

IBM Quantum Runtime service and Sampler examples
https://docs.quantum.ibm.com/api/qiskit-ibm-runtime/runtime_service

Qiskit Runtime migration notes for Sampler results
https://docs.quantum.ibm.com/migration-guides/qiskit-runtime-options

General-knowledge extension questions are marked in each topic. Qiskit package APIs evolve; match runtime-specific setup details to the installed version when using the notebooks.